

Technical Architecture

Runtime control for AI agents — internal documentation

Generated 2026-10-02 · source: verumsell.com/work/custos#how · English only

Custos sits inline between an AI agent and the MCP tools it calls: it identifies the agent, checks the call against policy, writes a tamper-evident record, and only then lets it through — or doesn't.

“Every decision the gateway makes is written and flushed to a tamper-evident, hash-chained log before the call is allowed through — not after.”

01

The gateway — the enforcement point

- Written in Rust (Tokio, axum, request): a single static binary, no runtime, no garbage-collector pauses on the request path — because this sits inline on every tool call an agent makes, so latency has to be predictable.
 - Speaks MCP's streamable-HTTP transport directly and proxies raw JSON-RPC bytes rather than implementing a full MCP SDK. It only needs to read the call's method and tool name to decide — which keeps it compatible as the MCP spec evolves underneath it.
 - Session binding: MCP sessions are tied to the agent that opened them. A second agent presenting the same session id gets rejected and is logged as a reuse attempt — it is never forwarded. Idle sessions expire and the session table is capped with LRU eviction, so a hostile client can't exhaust memory by opening sessions forever.
 - Tool-list filtering: an agent only ever sees the tools its policy would actually allow it to call — the tool list is rewritten before it reaches the agent. A response shape the gateway doesn't recognise fails closed rather than passing through unfiltered.
 - Credential isolation: only headers on an explicit allow-list go upstream. The agent's own bearer token never reaches the MCP server — upstream credentials come from the gateway's own config, set once by the operator.
-

02

Policy: Cedar, not a hand-rolled rules engine

- Policies are written in Cedar, AWS's open-source authorization language — chosen over something like OPA/Rego because it's purpose-built for authorization and designed for formal analysis. A roadmap item is answering “can any agent ever reach payroll?” by mathematical proof, not just by testing.
 - Default deny, structurally: a call is allowed only if some permit rule matches and no forbid rule does. There is no code path that defaults to allow.
 - Schema-validated: policies are checked against a schema on load and on every reload. An unknown attribute in a policy is a validation error, not a silent no-op.
 - Hot reload without dropping connections: a reload is fully validated before it replaces the running policy set. A bad reload leaves the previous, known-good set in charge, and a request already in flight finishes under the policy version it started with.
 - Every decision names the specific policy that made it, including “default deny” when nothing matched — so a blocked call always comes back with a reason, not a bare rejection.
-

03

Content inspection — looking inside the call

- A dependency-free module walks a tool call's arguments and flags what it finds, so policy can act on content, not just on which tool was named.
 - Detectors use real checksums, not just pattern matching: IBAN (mod-97), card numbers (Luhn), national ID formats, plus email addresses, API-key and secret patterns, and bulk-size signals such as argument size and array length.
 - Bounded walk: nested JSON is walked depth- and byte-capped by config, so an oversized or hostile payload fails closed instead of slowing the gateway down.
 - Findings never carry the matched value — only kind, count and location. A finding that an argument “looks like an IBAN” never stores which IBAN.
 - Findings feed back into policy as context, so a rule can say “forbid any call whose findings contain a card number or secret” or “forbid CRM exports over 500 rows” — behavioural policy, not just allow-lists.
-

04

The audit log — tamper-evident, not just a log file

- Append-only, SHA-256 hash-chained: every record's hash covers the previous record's hash, so an edit, insertion or deletion anywhere in the file breaks the chain from that point forward. A verify command walks the whole file and proves — or disproves — integrity in one pass.
 - Audit-before-act: the decision record is written and flushed to disk before the call is forwarded. If the write fails, the call is blocked — there is no path where a call proceeds without a durable record of why.
 - Three privacy modes, one gateway-wide setting: hash (HMAC-SHA256 of the arguments, the default), redacted (same shape, every value replaced by its type and length), or full (arguments as-is, with a startup warning since this may store personal data).
 - Hash mode uses HMAC rather than a plain hash deliberately — a plain hash of a short, guessable value like an IBAN or national ID can be brute-forced offline. HMAC mixes in a secret key, so a guess is useless without it. If hash mode is selected and the key is missing, the gateway refuses to start rather than silently falling back to something weaker.
 - Each record also carries the agent's configured owner, the gateway instance, and the exact policy version that made the decision — enough to answer “who owned this agent, which gateway, under which policy” for any single row, months later.
-

05

Tokens and identity

- Two auth modes, operator's choice: static hashed tokens (SHA-256, simple setups) or signed, short-lived tokens (Ed25519, carrying agent id, issued-at, expiry and key id).

- Key rotation is built in — the gateway accepts a list of verifying keys by key id, so an old key keeps validating already-issued tokens while a new one takes over issuing.
 - Negative cases are tested explicitly: an expired token, a wrong key and a tampered payload are each rejected.
 - Plain tokens are never held anywhere, by design — only hashes, or signed tokens that are verified, never decrypted back into something secret.
-

06

Custos Control — the management plane

- Self-hosted first: Control and its database run entirely inside the customer's own network — a direct answer to the question security buyers raise first, “where does our audit data live.” Every table carries a tenant id from day one, so a future hosted option is the same schema, not a rewrite.
 - Gateways connect out to Control, never the other way — a customer opens zero inbound ports to make this work.
 - Control never holds agent tokens in plaintext and never sees raw tool arguments beyond whatever the gateway's own privacy mode already allows through.
 - Policy bundles are signed, not just transmitted: publishing a policy version produces one signed bundle — policy text, schema, and the exact active-agent set at publish time. A tampered bundle fails signature verification and is rejected outright.
 - A gateway that loses contact with Control keeps enforcing its last known-good policy — it never falls back to “allow all.” Connecting to Control is optional: no configuration means zero outbound calls and identical behaviour to a standalone gateway.
 - Audit logs round-trip too: gateways ship their hash-chained log to Control in batches, idempotent by sequence number so a retried batch is a no-op, not a duplicate. Control checks chain continuity per gateway and flags gaps — a flag never silently clears itself.
-

07

Human-in-the-loop: hold and four-eyes approval

- A policy can mark a rule as requiring hold: matching calls become a pending approval instead of an automatic allow.
- The gateway raises the approval in Control and polls for a decision up to a timeout. Approved means forward; rejected, timed out, or Control unreachable all mean block. Every step — the hold, the eventual allow or block — is written to the audit log before it's acted on.
- Four-eyes: the person who approves a held call can't be the same person who owns the agent that made it, enforced at resolution time — and it fails closed (blocks) if Control has no record of the agent at all, rather than silently skipping the check.

- Approvals are role-gated to designated approvers, enforced server-side, not just hidden in the dashboard UI.

08

Evidence export — audit trail to compliance artifact

- One export gathers a time-ranged pack — agent inventory, published policy versions, decision statistics, resolved approvals with approver identities, per-gateway chain-integrity status — and produces both a signed JSON bundle and a human-readable PDF, generated without a headless browser or external binary.
- Localised in English, German and Romanian, including the diacritics each language needs.
- Maps specific controls to named articles — EU AI Act Art. 12 & 14, NIS2 Art. 21(2)(a)/(e), GDPR Art. 5(1)(c)/(f) — worded as the control “supports” the requirement, never that it “satisfies” or “complies with” one. It’s our own reading of the regulations: useful groundwork for your legal and compliance team, not a substitute for their review.

09

Security invariants

The rules the whole system is built around — each enforced in code, and each with a test that proves the negative case, that a blocked call never reaches the upstream tool, not just that the log says it was blocked.

- Fail closed. Anything unparseable, unauthenticated, unevaluated or unauditable is rejected — there is no “allow on error” path anywhere.
- Default deny. Allowed only with a matching permit and no matching forbid.
- Audit before acting. The decision is durably written before the call is forwarded; a failed write blocks the call.
- Credentials never cross the boundary. Only an explicit header allow-list goes upstream; the agent’s own token never does.
- No plaintext tokens, anywhere, ever — SHA-256 hash or verified signature only.
- Hostile input is the default assumption. Tool names, arguments, request ids, upstream responses and config values are all treated as untrusted.
- No unsafe code paths outside tests — enforced at the workspace level, not left to code review.
- Every security behaviour has a test, including its failure mode.

10

The stack, end to end

LAYER	CHOICE	WHY
Gateway, Control API	Rust, Tokio, axum, request	memory safety, no GC pause, single static binary
Policy	Cedar (cedar-policy)	built for authorization, fast, analyzable
Tokens	SHA-256 (static) / Ed25519 (signed, short-lived)	no reversible secret storage
Audit	Hash-chained JSON Lines + HMAC-SHA256	tamper-evident, brute-force-resistant
Config / tenants (Control)	PostgreSQL	self-hostable, well-understood, tenant id from day one
Dashboard	Vite + React + TypeScript, embedded in the binary	one process to run, not a constellation of services
PDF generation	Pure-Rust PDF rendering	no headless browser, no outbound dependency
Deploy	Distroless, non-root image; EU (Frankfurt) hosted option	small attack surface; self-hosted-first for data residency

11

Built today, and what's next

BUILT TODAY

- + Full gateway enforcement path, end to end
- + Cedar policy with schema validation and hot reload
- + Content inspection feeding live policy context
- + Hash-chained audit log with all three privacy modes
- + Both token schemes, with key rotation
- + The full Control plane: agents, signed policy bundles, gateway sync, audit ingestion with chain flagging, hold/four-eyes approval
- + The dashboard and its main pages
- + Evidence export via API (a dashboard button is next)

ON THE ROADMAP

- OIDC login for the dashboard
- A hosted, multi-tenant EU Control option
- An LLM API egress proxy
- Formal policy analysis — provable reachability questions, not just tests

- Approval notifications (Slack, Teams, email)